Alexandria University
Faculty of Engineering
Computer and Systems Engineering Department

CSE: Computer Vision

Assignment 1

Fine Detail and the Big Picture

Report

Team member	ID
Omar Khaled Hussein	22010962
Yaseen Asad Ahmed	22011349
Abdelrahman Khaled Nour	22010877

Assigned: Fri, Oct 9, 2026 Due: Mon, Oct 19, 2026

Language: Python (NumPy, SciPy for verification only)

Contents

1 Part 1: Filters and Edges **1**
1.1 Convolution from Scratch . . . 1
1.2 Finite Difference Operator . . . 3
1.3 Derivative of Gaussian (DoG) Filter . . . 4

2 Part 2: Frequencies, Pyramids, and Blending **6**
2.1 Hybrid Images . . . 6
2.2 Aliasing and Gaussian Pre-Filtering . . . 10
2.3 Gaussian and Laplacian Pyramids . . . 12
2.4 Multiresolution Image Blending . . . 14

3 Part 3: Seam Carving **17**
3.1 Finding and Removing Seams . . . 17
3.2 Results and Comparisons . . . 19

4 Bonus: Object Removal and Width Restoration **23**
4.1 Removing an Object . . . 23
4.2 Restoring the Width by Seam Insertion . . . 25
4.3 Results . . . 27

References **29**

A Implementation Notes **29**
A.1 Padding Conventions . . . 29
A.2 Separable Convolution via SVD . . . 29
A.3 Binomial vs. Gaussian Filter in Pyramids . . . 29
A.4 Reproducibility . . . 29

1 Part 1: Filters and Edges

1.1 Convolution from Scratch

What we did

We implemented 2D convolution for single-channel images in three versions. All of them use zero padding so that the output has the same size as the input, and all of them flip the kernel along both axes (convolution is correlation with the flipped filter).

- **Loop version** (`convolve_loop`): four nested loops over the output pixels and the filter entries.
- **Vectorized version** (`convolve_vectorized`): no loop over pixels. The padded image is viewed as all $k_h \times k_w$ windows with `sliding_window_view`, and one `einsum` contracts every window with the flipped kernel in a single whole-image operation.
- **Separable version** (`convolve_separable`): computes the SVD of the kernel and tests whether it has rank 1 up to rounding error ($\sigma_2 \leq 10^{-10}\sigma_1$). If so, the kernel is written as column $\times$ row, and the image is convolved with the column and then with the row. Otherwise it falls back to the vectorized version.

Code

```python
def _as_float_inputs(image, kernel):
    """Validate grayscale shapes and return floating-point arrays."""
    image = np.asarray(image, dtype=np.float64)
    kernel = np.asarray(kernel, dtype=np.float64)
    if image.ndim != 2 or kernel.ndim != 2:
        raise ValueError("image and kernel must both be 2D arrays")
    if image.size == 0 or kernel.size == 0:
        raise ValueError("image and kernel must not be empty")
    if kernel.shape[0] % 2 == 0 or kernel.shape[1] % 2 == 0:
        raise ValueError("kernel dimensions must be odd")
    return image, kernel


def convolve_loop(image, kernel):
    """Convolve a grayscale image with loops over pixels and kernel entries."""
    image, kernel = _as_float_inputs(image, kernel)
    height, width = image.shape
    kernel_height, kernel_width = kernel.shape
    pad_y, pad_x = kernel_height // 2, kernel_width // 2
    padded = np.pad(image, ((pad_y, pad_y), (pad_x, pad_x)), mode="constant")
    # Convolution flips the kernel in both dimensions before applying it.
    flipped_kernel = np.flip(kernel, axis=(0, 1))
    output = np.zeros((height, width), dtype=np.float64)
    for image_y in range(height):
        for image_x in range(width):
            for kernel_y in range(kernel_height):
                for kernel_x in range(kernel_width):
                    output[image_y, image_x] += (
                        padded[image_y + kernel_y, image_x + kernel_x]
                        * flipped_kernel[kernel_y, kernel_x]
                    )
    return output


def convolve_vectorized(image, kernel, padding_mode="constant"):
    """Convolve using NumPy operations without loops over image pixels."""
    image, kernel = _as_float_inputs(image, kernel)
    pad_y, pad_x = kernel.shape[0] // 2, kernel.shape[1] // 2
```

```
    padded = np.pad(image, ((pad_y, pad_y), (pad_x, pad_x)), mode=padding_mode)
    windows = np.lib.stride_tricks.sliding_window_view(padded, kernel.shape)
    flipped_kernel = np.flip(kernel, axis=(0, 1))
    return np.einsum("ijkl,kl->ij", windows, flipped_kernel)


def convolve_separable(image, kernel, rank_tolerance=1e-10,
                       padding_mode="constant"):
    """Use two 1D convolutions for rank-1 kernels; otherwise use vectorized."""
    image, kernel = _as_float_inputs(image, kernel)
    left, singular_values, right = np.linalg.svd(kernel, full_matrices=False)
    is_rank_one = (
        singular_values.size == 1
        or singular_values[1] <= rank_tolerance * singular_values[0]
    )
    if not is_rank_one:
        return convolve_vectorized(image, kernel, padding_mode=padding_mode)
    # SVD writes a rank-1 kernel as column @ row; convolve each factor in turn.
    column = left[:, 0] * singular_values[0]
    row = right[0, :]
    result = convolve_vectorized(image, column[:, np.newaxis],
                                 padding_mode=padding_mode)
    return convolve_vectorized(result, row[np.newaxis, :],
                               padding_mode=padding_mode)
```

Sanity check

We compared the three versions against `scipy.signal.convolve2d` (`mode="same"`, zero fill) on small random images (3×4 and 7×6) with random odd-sized filters of shapes 1×1, 1×5, 5×1, 3×5, 5×3 and 3×3. For every shape we tested both a random (non-separable) kernel and a separable one (outer product of a random column and a random row). All three versions match SciPy to rtol = atol = 10^{-10}. We also verified that the inputs are not modified and that even-sized kernels are rejected.

```
Running Part 1.1 correctness tests...
test_even_sized_kernel_is_rejected ................ ok
test_inputs_are_not_modified ...................... ok
test_random_odd_kernels_match_scipy ............... ok

Ran 3 tests in 0.024s OK
```

Figure 1: Output of the Part 1.1 correctness tests.

Timing

Setup: a 256×256 random image, normalized square Gaussian kernels with $\sigma = \text{size}/6$, one run per method and kernel. The separable time includes the SVD rank test.

Kernel	Loop (ms)	Vectorized (ms)	Separable (ms)	Loop / Vec.	Vec. / Sep.
3×3	215.42	1.51	0.85	143×	1.8×
5×5	587.81	1.77	0.67	332×	2.6×
7×7	1052.28	3.23	0.80	326×	4.0×
15×15	4702.00	21.48	2.43	219×	8.8×
31×31	37527.89	70.37	6.38	533×	11.0×

Table 1: Running time of the three convolution versions on a 256×256 image (single run each).

Explanation of the differences

- **Loop.** It performs $O(HWk^2)$ multiply-adds, and every one is executed one at a time by the Python interpreter (about 0.5 μs each). Its time therefore grows roughly with k^2: from 3×3 to 5×5 the number of taps grows by 2.8×, and the time grows by 2.7×.
- **Vectorized.** It performs the same $O(HWk^2)$ arithmetic, but inside compiled NumPy code over whole arrays, with no interpreter overhead per pixel. It is between 143× and 533× faster than the loop. For small kernels, fixed costs (padding, creating the window view) dominate, which is why the time barely grows from 3×3 to 5×5.
- **Separable.** A Gaussian is the product of a column and a row (rank 1), so two 1D passes cost $O(HW \cdot 2k)$ instead of $O(HWk^2)$. The ideal speed-up over the vectorized version is $k/2$, so it grows with the kernel size: 11× at 31×31 (the ideal is ≈ 15.5×; the gap comes from the two passes, the temporary array and the SVD that is included in the time). At 3×3 the gain is small because overhead dominates.
- **Fallback.** A non-separable filter is detected by the SVD test and handled by the vectorized version, so the separable function is always correct, and only sometimes faster.

1.2 Finite Difference Operator

What we did

Using our convolution from Section 1.1 (separable version, zero padding), we filtered a grayscale image (`facade.png`, intensities in $[0, 1]$) with $D_x = \begin{bmatrix} 1 & 0 & -1 \end{bmatrix}$ and $D_y = D_x^{\mathsf{T}}$. We then computed the gradient magnitude $\sqrt{(\partial I/\partial x)^2 + (\partial I/\partial y)^2}$ and binarized it with a threshold of 0.2. The signed derivative maps are displayed with zero mapped to mid-gray.

```python
def finite_difference_edges(image, threshold=0.2):
    """Return zero-padded finite differences, gradient magnitude, edge mask."""
    image = np.asarray(image, dtype=np.float64)
    derivative_x = np.array([[1.0, 0.0, -1.0]])
    derivative_y = derivative_x.T
    dx = convolve_separable(image, derivative_x, padding_mode="constant")
    dy = convolve_separable(image, derivative_y, padding_mode="constant")
    gradient_magnitude = np.sqrt(dx ** 2 + dy ** 2)
    edges = gradient_magnitude >= threshold
    return dx, dy, gradient_magnitude, edges
```

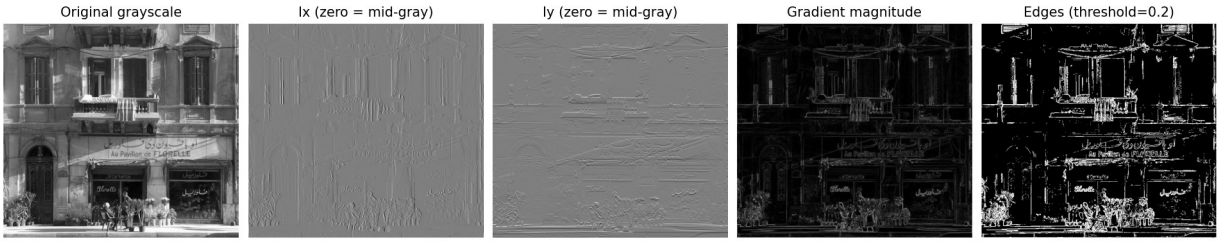


Figure 2: Part 1.2 on `facade.png`: original image, I_x, I_y (zero = mid-gray), gradient magnitude, and the edge image with threshold 0.2. Edge pixels: 64 616.

Observations

I_x responds to vertical structures (window frames, pillars, sides of the door) and I_y to horizontal ones (balcony rail, cornices, edges of the sign). The sign convention is correct: a step from dark to bright along x gives a positive I_x (verified by a unit test on a step image).

Question: justify your threshold

The image has intensities in $[0, 1]$, and we used a threshold of 0.2 on the gradient magnitude. A threshold that is too low keeps wall texture, plants and noise as edges, because differentiation amplifies every small intensity change. A threshold that is too high breaks the real contours into pieces. At 0.2 the main structures stay connected (windows, balcony, sign, door), and the edge map still contains the clutter at the base of the building, which is the expected price of an unsmoothed derivative that Section 1.3 reduces.

Question: where do the lines along the image frame come from?

They come from zero padding. The convolution treats everything outside the image as black (intensity 0). At the first column, $I_x = I[:, 1] - 0$, and at the last column $I_x = 0 - I[:, -2]$; the same holds for the first and last rows with I_y. The derivative at the border therefore equals the brightness of the neighboring pixel, so the border behaves like a step edge between the black padding and the image. This fake edge survives the threshold only where the border pixels are bright: it is strong along the bottom (bright pavement) and along the right side, and absent along the dark left side. Corner pixels receive both I_x and I_y, so their magnitude is up to $\sqrt{2}$ times larger. We confirmed this with a unit test on a constant image: with zero padding, $I_x = \pm 0.37$ at the first and last columns and $I_y = \pm 0.37$ at the first and last rows, and exactly zero everywhere else. With reflection padding these lines disappear, since the border derivative becomes zero.

1.3 Derivative of Gaussian (DoG) Filter

What we did

We built a normalized 2D Gaussian with $\sigma = 2$ (radius $\lceil 3\sigma \rceil = 6$, size 13×13). We then filtered the kernel itself with D_x and with D_y using a *full* convolution, so that nothing is truncated; the two results are the DoG filters, of sizes 13×15 and 15×13. Each DoG filter is applied to the image with a single convolution, and we repeat the steps of Section 1.2 (gradient magnitude, then threshold).

```python
def gaussian_kernel(sigma, size=None):
    radius = int(np.ceil(3 * sigma))
    size = 2 * radius + 1 if size is None else size
    coordinates = np.arange(size, dtype=np.float64) - size // 2
    one_dimensional = np.exp(-(coordinates ** 2) / (2 * sigma ** 2))
    one_dimensional /= one_dimensional.sum()
    return np.outer(one_dimensional, one_dimensional)


def convolve_full(image, kernel):
    """Full 2D convolution using only NumPy operations."""
    kh, kw = kernel.shape
    padded = np.pad(image, ((kh - 1, kh - 1), (kw - 1, kw - 1)),
                    mode="constant")
    windows = np.lib.stride_tricks.sliding_window_view(padded, kernel.shape)
    return np.einsum("ijkl,kl->ij", windows, np.flip(kernel, axis=(0, 1)))


def derivative_of_gaussian_filters(sigma, size=None):
    gaussian = gaussian_kernel(sigma, size=size)
    derivative_x = np.array([[1.0, 0.0, -1.0]])
    derivative_y = derivative_x.T
    dog_x = convolve_full(gaussian, derivative_x)
    dog_y = convolve_full(gaussian, derivative_y)
    return gaussian, dog_x, dog_y


```

```python
def derivative_of_gaussian_edges(image, sigma=2.0, threshold=0.2, size=None):
    gaussian, dog_x, dog_y = derivative_of_gaussian_filters(sigma, size=size)
    ix = convolve_separable(image, dog_x, padding_mode="constant")
    iy = convolve_separable(image, dog_y, padding_mode="constant")
    gradient_magnitude = np.sqrt(ix ** 2 + iy ** 2)
    edges = gradient_magnitude >= threshold
    return gaussian, dog_x, dog_y, ix, iy, gradient_magnitude, edges
```

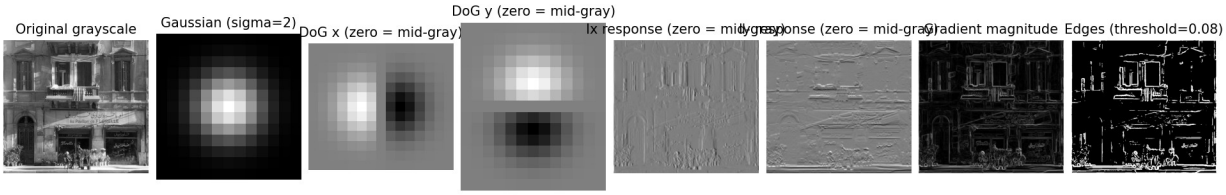


Figure 3: Part 1.3 ($\sigma = 2$, threshold 0.08): original image, Gaussian, DoG in x, DoG in y (zero = mid-gray), the two filter responses I_x and I_y, the gradient magnitude, and the edge image. Edge pixels: 57 697.

Summary

	1.2 Finite differences	1.3 DoG
Filter	$\begin{bmatrix}1 & 0 & -1\end{bmatrix}$	DoG, $\sigma = 2$, 13×15 and 15×13
Convolutions per direction	1	1
Threshold	0.2	0.08
Edge pixels	64 616	57 697

Table 2: Comparison of the two edge detectors on the same image.

Question: state your σ and threshold

We used $\sigma = 2$ and a threshold of 0.08. A larger σ suppresses more noise but blurs the edges and loses localization, and a smaller σ approaches the plain finite difference. Blurring lowers the gradient magnitudes, so the same threshold as in Section 1.2 (0.2) would remove most edges. We therefore lowered it to 0.08, so that the two edge maps have a comparable number of edge pixels (64 616 against 57 697) and the comparison is fair.

Question: how and why do the maps differ from those of Section 1.2?

How. The Section 1.3 maps are smoother. The edges are longer and more continuous, and the fine texture of the wall, the plants and small noise is largely gone. The main structures (windows, balcony, sign outline, door arch) are present in both, but their lines are thicker here. Fine details such as the small Arabic lettering and the plants merge or are lost. The overall gradient magnitude is lower, which is why the threshold had to be lowered. The frame response from zero padding is still present, but it is now a band about as wide as the kernel radius (6 pixels) instead of 1 pixel.

Why. Differentiation amplifies high frequencies, which is where noise and fine texture live, so the finite difference of Section 1.2 reacts to every small variation. In Section 1.3 the image is blurred before it is differentiated, which removes these high frequencies before they can be amplified. Because convolution is associative,

$$\frac{\partial}{\partial x}(G * I) = \left(\frac{\partial G}{\partial x}\right) * I,$$

so a single DoG filter performs the blur and the derivative together with one convolution. The price is localization: the blur spreads each edge over roughly σ pixels, so edges become thicker and less precise. Finally, the DoG filters are also separable (a Gaussian column times a row filtered with D_x), so they use the fast path of our `convolve_separable`.

2 Part 2: Frequencies, Pyramids, and Blending

2.1 Hybrid Images

Method

A hybrid image [1] is constructed by adding the low-pass version of one image to the high-pass version of a second:

$$H = \underbrace{G_{\sigma_\ell} * I_1}_{\text{low-pass}} + \underbrace{I_2 - G_{\sigma_h} * I_2}_{\text{high-pass}}, \tag{1}$$

where G_σ is a Gaussian of standard deviation σ and "$*$" denotes 2D convolution. At close viewing distance the eye resolves fine detail and perceives I_2; at a distance only coarse structure is visible and the viewer perceives I_1.

Both Gaussians use reflected padding (`padding_mode="reflect"`), implemented in `hybrid_images.py` via `convolve_separable` (Section 1.1). The second image is aligned to the first by two corresponding point pairs (e.g. left-eye and right-eye centres), determining a similarity transformation (scale, rotation, translation).

All three pairs use $\sigma_\ell = 4$, $\sigma_h = 4$ (values chosen by visual inspection of the hybrid).

Pair 1 — Man / Cat (required starter pair)

Low-pass image: Man portrait (`hybrid_man.jpg`). **High-pass image:** Cat (`hybrid_cat.jpg`). **Alignment points:** left eye $(299, 341)$ / $(443, 332)$ in the man; $(605, 290)$ / $(760, 372)$ in the cat.



(a) Man (low-pass input).



(b) Cat (aligned, high-pass input).



(c) Hybrid result.



(d) Low-pass filtered man ($\sigma_\ell = 4$).



(e) High-pass filtered cat ($\sigma_h = 4$).

Figure 4: Man → Cat hybrid (pair 1). Close up: the cat's fur texture and whiskers dominate. At distance: the man's blurred portrait is perceived.

Pair 2 — Cat / Tiger

Low-pass: Domestic cat (`cat.jpg`). **High-pass:** Tiger (`Tiger.jpg`). Alignment on matching eye centres.



(a) Cat (low-pass input).



(b) Tiger (aligned, high-pass input).



(c) Hybrid result.



(d) Low-pass filtered cat.



(e) High-pass filtered tiger.

Figure 5: Cat → Tiger hybrid (pair 2). The shared facial geometry (eye spacing, snout width, ear placement) makes the transition especially convincing.

Pair 3 — Dodge Challenger / Kawasaki Motorcycle

Low-pass: Dodge Challenger (front view). **High-pass:** Kawasaki Ninja ZX-10RR (front view). Alignment on front-wheel-centre and headlight correspondence.



(a) Dodge Challenger (low-pass input).



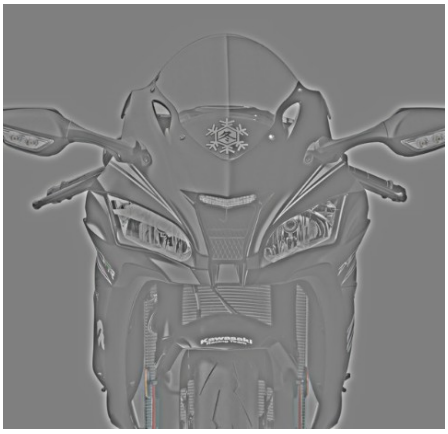
(b) Kawasaki (aligned, high-pass input).



(c) Hybrid result.



(d) Low-pass filtered Dodge.



(e) High-pass filtered motorcycle.

Figure 6: Dodge → Kawasaki hybrid (pair 3). Close up: the motorcycle's fairing, mirror and headlight outlines are visible. At distance: the wide-body muscle-car silhouette dominates.

Favourite pair — Man / Cat with Fourier analysis

The man–cat pair is selected as the favourite because the facial geometry aligns closely, producing a compelling multi-scale percept. Figure 7 shows all five spatial stages and their centred log-magnitude Fourier spectra.

How to read the spectra. Each spectrum shows $\log(1 + |\hat{f}(u, v)|)$ centred at DC (zero frequency). Bright regions indicate high energy at those spatial frequencies. The faint horizontal and vertical cross visible in several spectra is a boundary effect of the DFT (it treats the image as periodic, so the discontinuity between opposite borders leaks energy along the two axes) and not image content.

- **Man (a):** energy at all frequencies, concentrated at the centre and decaying outwards in a star-like pattern that follows the dominant edge orientations of the face.
- **Aligned cat (b):** a similar full-band spectrum, with oriented streaks that come from the fur texture and whiskers.
- **Low-pass man (c):** the outer frequencies are strongly attenuated (dark background); the Gaussian acts as a low-pass filter in the frequency domain and only the low-frequency core remains (plus the boundary cross).
- **High-pass cat (d):** the lowest frequencies are removed (dark spot at DC) and the remaining energy is spread over the mid and high frequencies, away from the centre.
- **Hybrid (e):** a bright low-frequency core inherited from the man, plus the cat's high-frequency streaks at larger radii. The logarithmic display compresses these high-frequency

components, so they look weak in the spectrum even though they dominate the close-up percept. This dual-band structure explains the perceptual switch: the visual system integrates low frequencies at distance (seeing the man) and resolves high frequencies close up (seeing the cat).

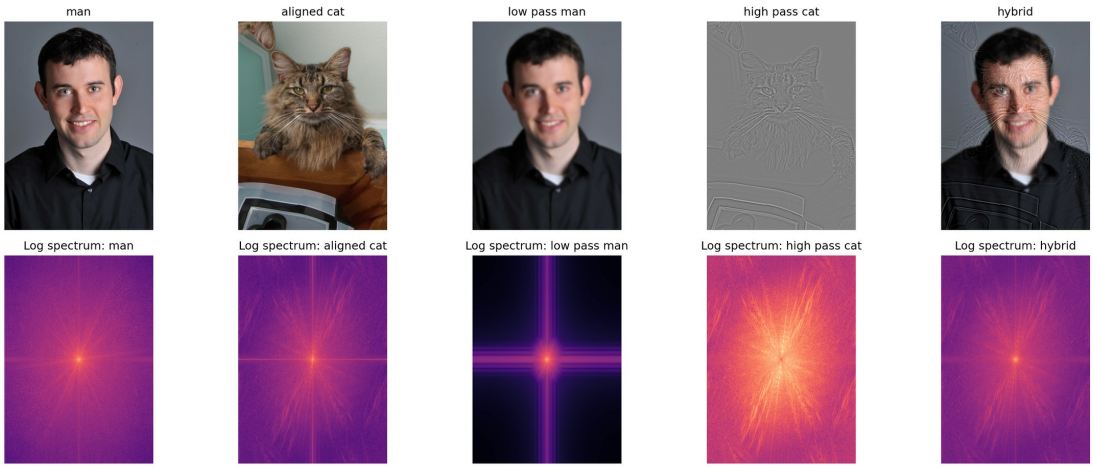


Figure 7: Combined image-and-spectrum overview for the favourite pair (man–cat). Top row: spatial images (a)–(e), left to right; bottom row: corresponding log-magnitude spectra.

2.2 Aliasing and Gaussian Pre-Filtering

Method

The Nyquist–Shannon sampling theorem states that downsampling by a factor of k requires all spatial frequencies above π/k to be removed first. Retaining them causes *aliasing*: high-frequency content folds into false lower-frequency patterns.

The experiment uses the concentric-rings test image `rings.png` (512×512 px). Rings of increasing spatial frequency move from centre to edge; those near the edge exceed the Nyquist limit of the $4\times$ downsampled output (128×128) and alias into conspicuous false rings.

Two strategies are compared:

1. **Direct downsampling:** keep every 4th row and column with no pre-filter; implemented in `aliasing.py` as `downsample_by_factor(image, factor=4)`.

2. **Gaussian pre-filtering:** apply G_σ first, then downsample; frequencies above $\pi/4$ are attenuated before sampling. Implemented as `aliasing_comparison(image, sigma, factor=4)`.

The optimal σ is found by a visual sweep over

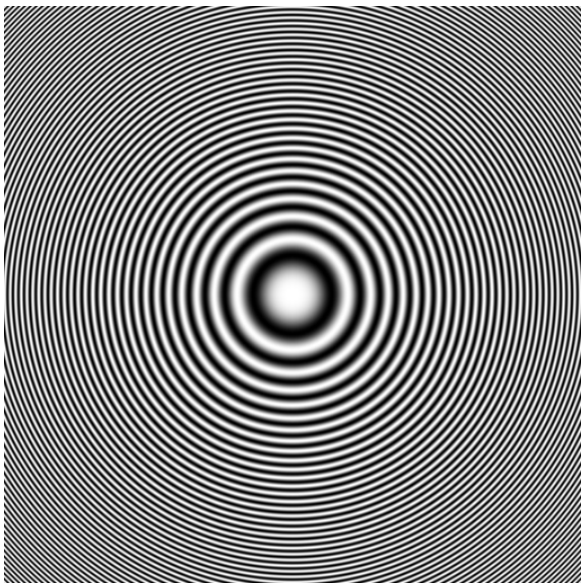
$$\sigma \in \{0.50, 0.75, 1.00, 1.25, 1.50, 1.75, 1.80, 1.85, 1.90, 1.95, 2.00, 2.25\}.$$

Results

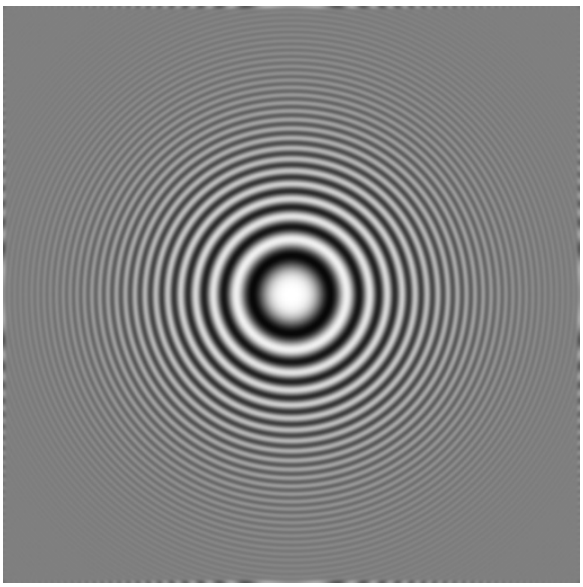
Discussion

Chosen sigma: $\boldsymbol{\sigma = 2.0}$ (smallest tested value without visible false circles). This is a subjective, image- and viewing-scale-dependent criterion.

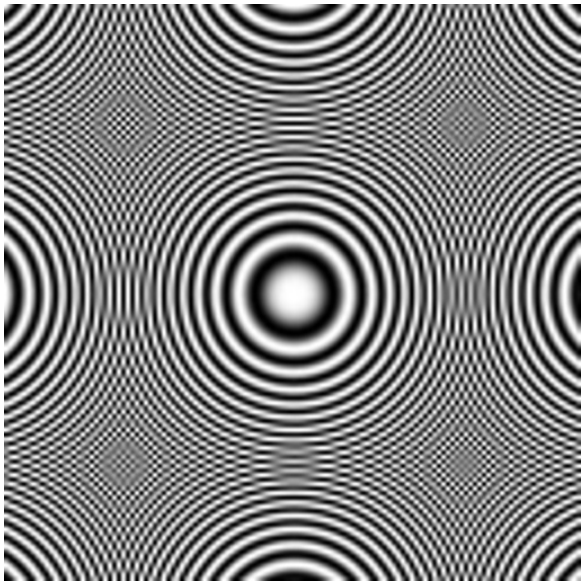
Why aliasing occurs. Direct downsampling keeps one sample out of every 4×4 block without removing frequencies above the new Nyquist limit $\pi/4$. A ring with spatial frequency $\pi/4 < f < 3\pi/4$ appears in the output at the alias frequency $|f - \pi/2|$, producing a false ring at a different period. The result is the conspicuous false circles visible in Figure 8(c).



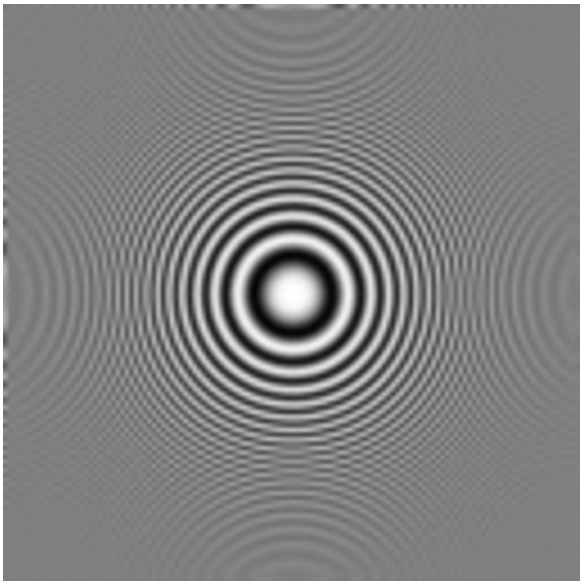
(a) rings image (512×512).



(b) Gaussian-blurred image ($\sigma = 2.0$).



(c) Direct $4\times$ **downsample** (128×128). Strong false circles visible in the corners — aliasing artefacts.



(d) Pre-filtered then downsampled ($\sigma = 2.0$, 128×128). Aliasing suppressed; only resolvable rings remain.

Figure 8: Aliasing comparison on the concentric-rings test image.

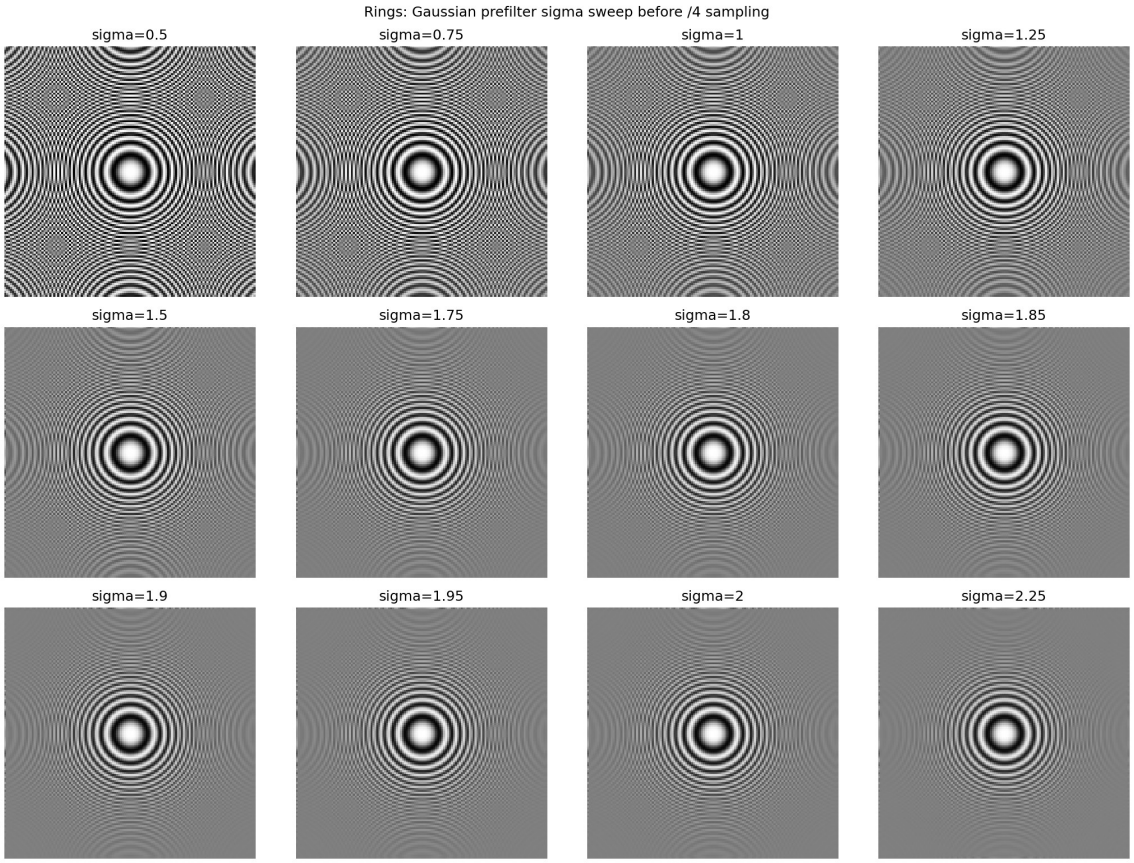


Figure 9: Sigma sweep ($\sigma \in \{0.5, \ldots, 2.25\}$, 4× downsample). False circles gradually disappear as σ increases. At $\sigma = 1.95$ faint residual structure is still visible; at $\sigma = 2.0$ it disappears at the inspected display scale.

Why Gaussian pre-filtering helps. The Gaussian is a low-pass filter: its frequency response is $e^{-\sigma^2\omega^2/2}$, so it attenuates energy above its cut-off before sampling. With $\sigma = 2.0$ the gain is about 0.29 at the new Nyquist frequency $\omega = \pi/4$ and about 0.007 at $\omega = \pi/2$, so most of the energy that would alias is removed; a small residual remains, which is why faint structure is still visible for σ slightly below 2 (Figure 9). The pre-filtered output (Figure 8(d)) shows essentially only rings that are resolvable at the output resolution, with no visible artefacts at the inspected scale. A larger σ would remove more aliasing, at the price of also blurring the rings that can be resolved.

Connection to Gaussian pyramids. Each `reduce()` step in Section 2.3 applies exactly the same principle: blur with the binomial filter (a close approximation to a Gaussian, with $\sigma = 1$, which suits a factor-2 reduction) and then downsample. This ensures each pyramid level is free of aliasing artefacts introduced by the coarser sampling grid.

2.3 Gaussian and Laplacian Pyramids

Method

Reduce. The binomial filter $w = \frac{1}{16}[1, 4, 6, 4, 1]$ is applied separably (horizontally then vertically); every other row and column is kept:

$$\text{reduce}(I) = \big(ww^{\mathsf{T}} * I\big)[\text{even rows, even cols}]. \tag{2}$$

Expand. Each sample is placed at even row/column positions of a zero-filled grid of double size; the binomial filter is applied and the result is scaled by 4 to compensate for the zero-insertion

energy loss:

$$\text{expand}(I) = 4\big(ww^{\mathsf{T}} * I{\uparrow}2\big). \tag{3}$$

Gaussian pyramid. $G_0 = I$; $G_{k+1} = \text{reduce}(G_k)$, continuing until the smaller side first drops to 16 pixels or fewer.

Laplacian pyramid.

$$L_k = G_k - \text{expand}(G_{k+1})[\text{cropped to } G_k\text{'s size}], \quad k = 0, \ldots, n-2; \qquad L_{n-1} = G_{n-1}. \tag{4}$$

Each L_k stores the detail not predicted by the next coarser level.

Reconstruction. $R_{n-1} = L_{n-1}$; $R_k = \text{expand}(R_{k+1})[\text{cropped}] + L_k$. Since $L_k \equiv G_k - \widetilde{G}_{k+1}$, summing recovers G_k exactly up to floating-point rounding.

Results

Image: `facade.png` (736×670 px — non-power-of-two dimensions).

Level	Width	Height	Pixels
0 (original)	736	670	493 120
1	368	335	123 280
2	184	168	30 912
3	92	84	7 728
4	46	42	1 932
5	23	21	483
6	12	11	132
Total			657 587

Table 3: Gaussian pyramid level dimensions (facade image, min_size = 16).

Pixel-count ratio: $657\,587/493\,120 \approx 1.334$, consistent with the geometric series $\sum_{k=0}^{\infty}(1/4)^k = 4/3 \approx 1.333$.

Reconstruction error: mean absolute error $= 7.67 \times 10^{-20}$; maximum absolute error $= 2.22 \times 10^{-16}$ (machine-precision rounding only).



Figure 10: Full Gaussian pyramid (7 levels, arranged largest to smallest). Each successive level is blurred and half-sized in both dimensions.

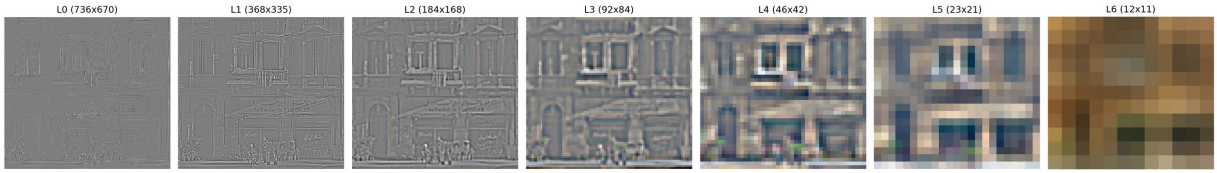


Figure 11: Full Laplacian pyramid. Each level stores a band-pass residual $L_k = G_k - \text{expand}(G_{k+1})$; the final level is the coarsest Gaussian. Values are shifted to $[0, 255]$ for display.



Figure 12: Left: original image; centre: reconstructed image; right: absolute error, contrast-boosted by dividing by its maximum value (2.22×10^{-16}). The error is at machine precision — reconstruction is effectively exact.

Discussion

Why reconstruction is exact. Each level $L_k = G_k - \widetilde{G}_{k+1}$ stores exactly the difference between the Gaussian level and its reconstruction predictor $\widetilde{G}_{k+1} = \text{expand}(G_{k+1})[\text{cropped}]$. Summing $\widetilde{G}_{k+1} + L_k$ recovers G_k algebraically; the only residual error is 64-bit floating-point rounding (MAE $< 10^{-19}$).

Why the lecture's blur-residual version is not exact. The lecture version subtracts a level's *own* blurred copy: $L_k^{\text{lec}} = G_k - \text{blur}(G_k)$. This residual *cannot* be used to recover G_k from G_{k+1} because $\widetilde{G}_{k+1} \neq \text{blur}(G_k)$ in general, so $\widetilde{G}_{k+1} + L_k^{\text{lec}} \neq G_k$.

Handling odd dimensions. `reduce` keeps $\lceil w/2 \rceil$ columns, so when G_k has odd width w, expand(G_{k+1}) has $2\lceil w/2 \rceil = w+1$ columns (for example $23 \to 12 \to 24$). Only the trailing row/-column is cropped; no internal pixels are discarded, and the reconstruction identity is preserved.

Pixel-count ratio. Levels at depth k contain $\approx (1/4)^k$ of the original pixels, giving a total that converges to $N \sum_{k=0}^{\infty} (1/4)^k = \frac{4}{3}N \approx 1.333N$. Truncating after 7 levels would give $1.3333N$; the measured value $1.3335N$ is marginally larger because the ceil-halving of the odd-dimensioned facade makes the coarse levels slightly bigger than $(1/4)^k N$.

2.4 Multiresolution Image Blending

Method

Multiresolution (Laplacian) blending [3] avoids seam artefacts by blending each frequency band of the Laplacian pyramid separately, using a mask that is *smoothed to match the band's spatial scale*:

$$B_k = M_k \cdot L_k^{(1)} + (1 - M_k) \cdot L_k^{(2)}, \tag{5}$$

where M_k is level k of the **Gaussian pyramid** of the binary mask, $L_k^{(1)}, L_k^{(2)}$ are the Laplacian pyramids of the two source images, and B_k is the blended Laplacian band. The blended image is then reconstructed from $\{B_k\}$.

At the finest level ($k = 0$), M_0 is close to the original binary mask, giving a sharp seam. At coarser levels, M_k is heavily Gaussian-blurred and the seam transitions over a wide area — matched to the coarser spatial wavelength of that band. The result hides the seam at *every* frequency.

Implementation (`blending.py`): `build_blended_laplacian_pyramid` constructs $L^{(1)}$, $L^{(2)}$ and the mask Gaussian pyramid; `multiresolution_blend` wraps it and calls `reconstruct_from_laplacian`.

Blend 1 — Apple / Orange (classic example)

Mask: vertical half-plane (left half selects apple, right half selects orange). **Pyramid:** 6 levels, 512×512 down to 16×16.

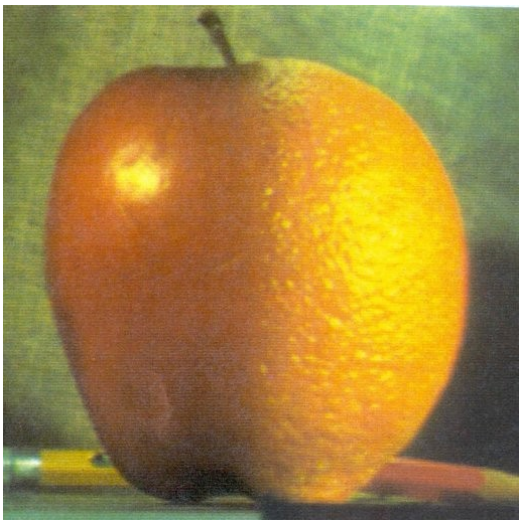


Figure 13: Apple–Orange **multiresolution blend**. The seam is invisible; colour and texture transition smoothly.

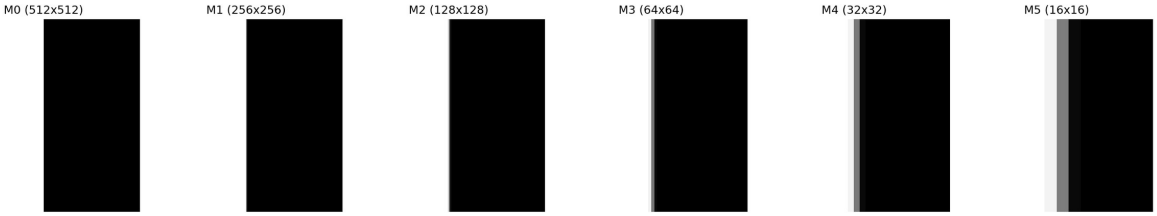


Figure 14: Mask Gaussian pyramid (transition widens with level).

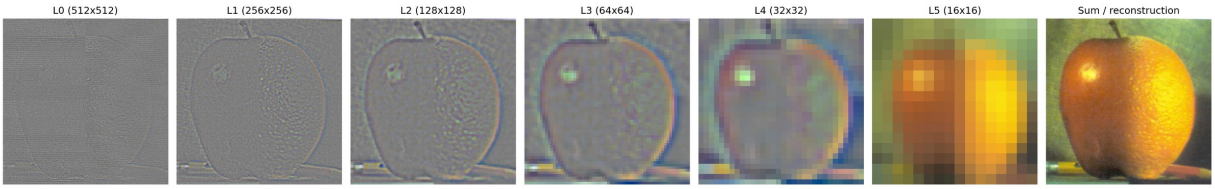


Figure 15: Blended Laplacian pyramid (B_k at each level; the last panel is the reconstructed sum).

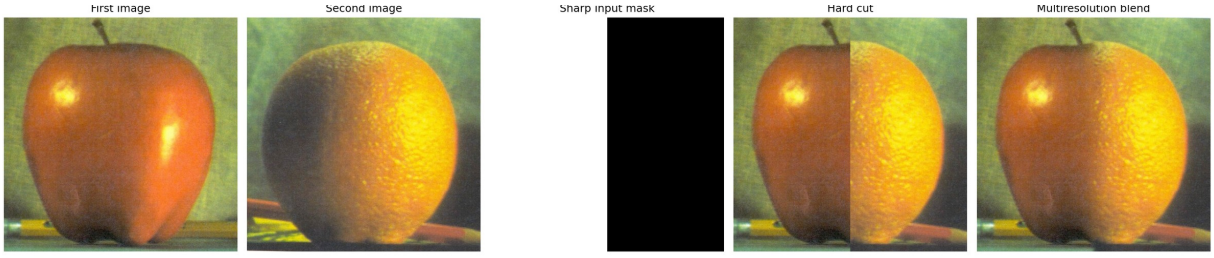


Figure 16: Apple–Orange full comparison: the two inputs, the sharp input mask, the hard cut and the multiresolution blend (right).

Blend 2 — Eye on Hand

Mask: binary ellipse centred on the palm. **Pyramid:** 7 levels, 900×675 down to 15×11. The eye is aligned to the palm using README-provided landmark points.

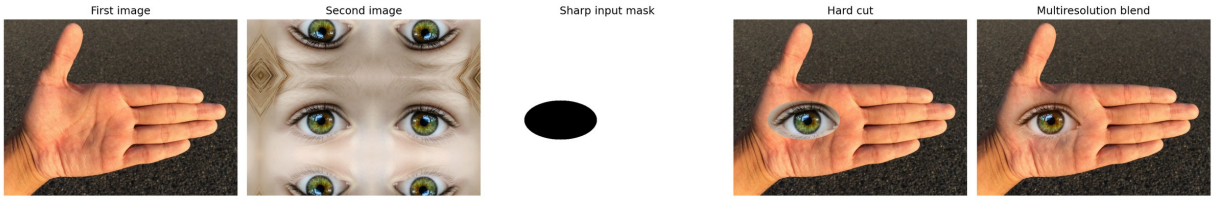


Figure 17: Eye-on-Hand: the two inputs, the sharp input mask, the hard cut and the multiresolution blend (right).

Blend 3 — Cat on Sunflower

Mask: three overlapping ellipses forming a head-and-ears silhouette. **Pyramid:** 6 levels, 612×408 down to 20×13. Cat eyes are aligned to the sunflower centre.



Figure 18: Cat-on-Sunflower: the two inputs, the sharp input mask, the hard cut and the multiresolution blend (right).

Blend 4 — Face on Apple (custom irregular mask)

Mask: three overlapping ellipses (head + two ear bulges) forming an irregular face-shaped mask. **Pyramid:** 6 levels, 512×512 down to 16×16. Portrait eyes are aligned across the apple.

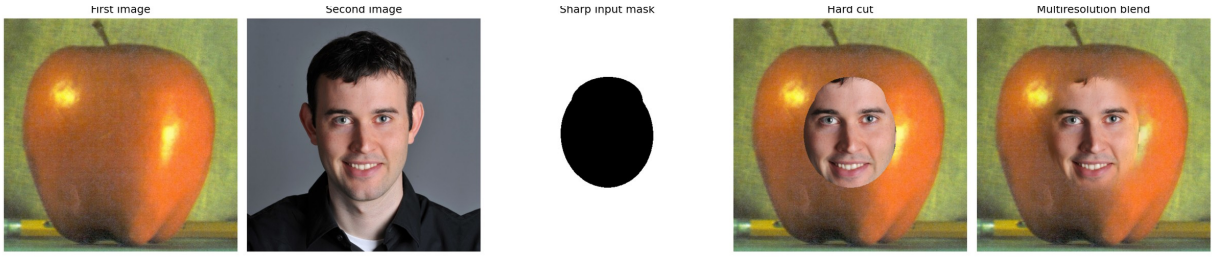


Figure 19: Face-on-Apple: the two inputs, the sharp input mask, the hard cut and the multiresolution blend (right).

Discussion — why multiresolution blending works

The fundamental problem with a hard cut. A sharp boundary has energy at *all* spatial frequencies. To hide a seam, the transition width must equal the spatial wavelength of each

frequency band: high frequencies need a *narrow* transition; low frequencies need a *wide* one. A single fixed blur width cannot achieve both simultaneously.

The multiresolution solution. The Gaussian pyramid naturally widens the mask's transition at each coarser level:

$$\text{transition width at level } k \;\propto\; 2^k \cdot \sigma_{\text{original}}.$$

Blending the Laplacian pyramid of each source image with this scale-dependent mask gives every frequency band its own appropriate transition width. The reconstructed composite therefore has no visible seam at *any* scale.

Comparison with hard cut. All four blends show the same pattern: the hard cut produces a stark colour or texture discontinuity at the mask boundary, while the multiresolution blend produces a smooth, natural-looking transition. The effect is most striking in the Apple–Orange blend, where the red-versus-orange colour difference would otherwise produce a vivid step edge.

3 Part 3: Seam Carving

3.1 Finding and Removing Seams

What we did

Seam carving [4] narrows an image by repeatedly removing the connected vertical path of pixels (the *seam*) that has the least energy.

- **Energy.** The gradient magnitude of Section 1.2, computed with our own filters (D_x, D_y through `convolve_separable`) on the grayscale version of the image. Edges have high energy, flat regions such as sky have low energy. Following the Part 2 rule (see Appendix A.1), the energy uses reflection padding, so the image border does not create a fake edge.
- **Seam.** One pixel per row, and each pixel touches the one above it, directly or diagonally (column change of -1, 0 or $+1$).
- **Dynamic program** (`find_vertical_seam`). We fill a table M row by row and trace the cheapest seam back from the bottom row (explained below).
- **Removal** (`remove_vertical_seam`). The seam pixel is deleted from every row, in all color channels. Then the energy is recomputed and the process repeats until the target width is reached (`carve_vertical`).
- **Original columns.** Alongside the image we keep an array holding each pixel's original column, and remove the same seams from it. Each removed seam is stored in original coordinates, so `draw_seams` can draw it where it sat in the original image.
- **Horizontal seams** (`carve_horizontal`). The image is transposed, vertical carving is applied, and the result is transposed back.

Code

```
def find_vertical_seam(energy):
    """Find a minimum-energy top-to-bottom seam by dynamic programming."""
    height, width = energy.shape
    cumulative = energy.copy() # M[i, j]
    predecessors = np.zeros((height, width), dtype=np.int64)
    columns = np.arange(width)
    for row in range(1, height):
        previous = cumulative[row - 1]
        from_left = np.full(width, np.inf)
        from_left[1:] = previous[:-1] # parent at (i-1, j-1)
```

```
        from_above = previous # parent at (i-1, j)
        from_right = np.full(width, np.inf)
        from_right[:-1] = previous[1:] # parent at (i-1, j+1)
        candidates = np.stack((from_left, from_above, from_right))
        best_predecessor = np.argmin(candidates, axis=0)
        cumulative[row] += candidates[best_predecessor, columns]
        predecessors[row] = columns + best_predecessor - 1
    seam = np.empty(height, dtype=np.int64)
    seam[-1] = int(np.argmin(cumulative[-1])) # cheapest end in last row
    for row in range(height - 1, 0, -1): # trace back to the top
        seam[row - 1] = predecessors[row, seam[row]]
    return seam


def remove_vertical_seam(image, seam):
    """Remove one connected vertical seam from a grayscale or RGB image."""
    height, width = image.shape[:2]
    keep = np.ones((height, width), dtype=bool)
    keep[np.arange(height), seam] = False
    remaining_shape = (height, width - 1) + image.shape[2:]
    return image[keep].reshape(remaining_shape)


def carve_vertical(image, seam_count, energy_function=compute_energy):
    """Remove vertical seams; return the carved image and original columns."""
    height, width = image.shape[:2]
    carved = image.copy()
    original_columns = np.tile(np.arange(width, dtype=np.int64), (height, 1))
    removed_seams = []
    rows = np.arange(height)
    for _ in range(seam_count):
        energy = np.asarray(energy_function(carved), dtype=np.float64)
        seam = find_vertical_seam(energy)
        removed_seams.append(original_columns[rows, seam].copy())
        carved = remove_vertical_seam(carved, seam)
        original_columns = remove_vertical_seam(original_columns, seam)
    seams = np.asarray(removed_seams, dtype=np.int64).reshape(seam_count, height)
    return carved, seams
```

Correctness tests

All 8 unit tests of `test_seam_carving.py` pass. The most important one compares the dynamic program with an exhaustive search: on a small 4×4 energy map it enumerates every connected path and checks that the cost of the seam found by the DP equals the minimum over all paths. The other tests check that seams are connected, that removing a seam keeps the remaining RGB pixels, that the original-column tracking stays in range, that horizontal carving reuses the vertical code, and that invalid inputs are rejected.

Result

Image: Broadway Tower, 714×484. We removed 30 % of the width, i.e. $\text{round}(0.30 \cdot 714) = 214$ seams, giving a 500×484 image.

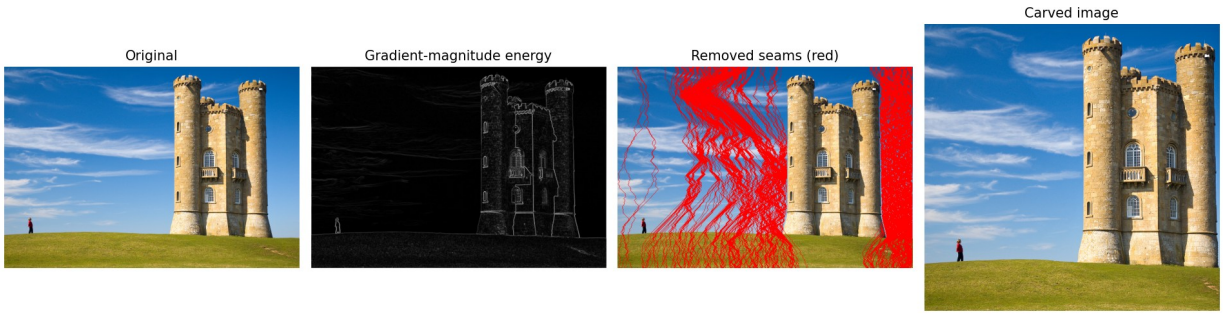


Figure 20: Part 3.1 on Broadway Tower. From left to right: the original, the gradient-magnitude energy, the 214 removed seams drawn in red on the original, and the carved 500×484 image.

The energy is high along the contours of the tower, the windows and the small figure, and almost zero in the sky and on the grass. The seams therefore run through the sky and bend diagonally across the grass, and they stay away from the tower and from the person. In the carved image the tower and the person keep their shape; only the empty sky between the left edge and the tower has been reduced.

Question: how does the dynamic program find the cheapest seam?

Let $E(i, j)$ be the energy and $M(i, j)$ the cost of the cheapest seam that ends at pixel (i, j), starting anywhere in the first row. Then

$$M(0, j) = E(0, j), \qquad M(i, j) = E(i, j) + \min\big\{M(i{-}1, j{-}1),\, M(i{-}1, j),\, M(i{-}1, j{+}1)\big\},$$

where a missing neighbor at the border counts as $+\infty$. We fill M row by row and store, for every pixel, which of the three parents gave the minimum. This works because the cheapest seam ending at (i, j) must continue the cheapest seam ending at one of its three parents: a cheaper way to reach the parent would give a cheaper way to reach (i, j). The smallest value in the last row of M is the cost of the best seam, and starting from that pixel we follow the stored parents back up to row 0 to read off the seam.

Question: why would trying every possible seam take forever?

Each seam pixel has up to three choices in the row below, so the number of connected seams grows exponentially with the height: roughly $W \cdot 3^{H-1}$. For our $H = 484$ that is about 10^{230} seams, far more than could ever be checked. The dynamic program computes every $M(i, j)$ once from three previous values, which costs $O(HW)$, here about $3.5 \cdot 10^5$ table entries per seam (about 10^8 in total for all 214 seams, including the energy recomputation).

3.2 Results and Comparisons

What we did

We reduced the width of six images by about 30 %: `rocket.jpg`, `home.jpg` and `starry_night.jpg` from the starter pack, and three photos of our own (a person in a field, a lake, and a group photo of five people). Images larger than 1000 pixels on the long side were first scaled down to a working size. For each image, three methods produce exactly the same width:

- **Normal resize:** `cv2.resize` (`INTER_AREA`) to the new width, keeping the height. This squeezes everything horizontally.
- **Center crop:** equal margins are cut from the left and right.
- **Seam carving:** our implementation of Section 3.1 with the finite-difference energy of Section 1.2.

The number of seams is round$(0.30 \cdot \text{width})$, and the target width is the width minus that number.

Image	Original	Seams removed	New width	Role
rocket	640×427	192	448	starter pack
home	512×384	154	358	starter pack
starry_night	720×568	216	504	starter pack
person outdoors	1000×667	300	700	own, shows a person
lake	1000×667	300	700	own
group photo	808×553	242	566	own, failure case

Table 4: Images used in Part 3.2 and the number of seams removed.

Comparison with resizing and cropping

Each figure shows the original and its three narrower versions: normal resize, center crop and seam carved.

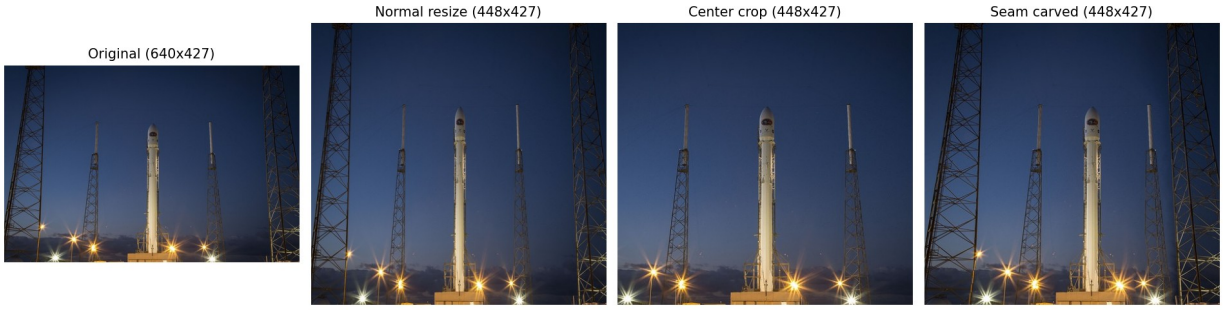


Figure 21: `rocket.jpg`: $640 \times 427 \rightarrow 448 \times 427$. Resize squeezes the rocket and the towers, the crop removes the outer towers, and seam carving keeps all of them.

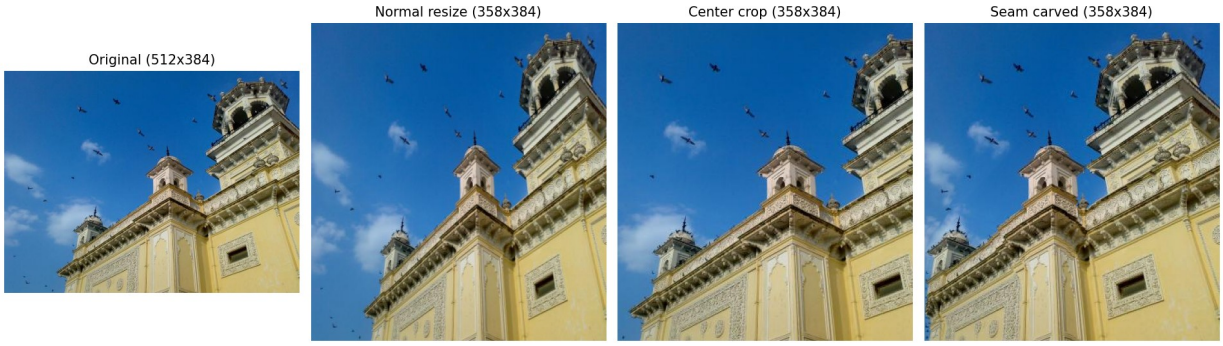


Figure 22: `home.jpg`: $512 \times 384 \rightarrow 358 \times 384$. Seam carving removes mostly sky, so the building keeps its proportions. The crop cuts part of the building off, and the resize squeezes it.

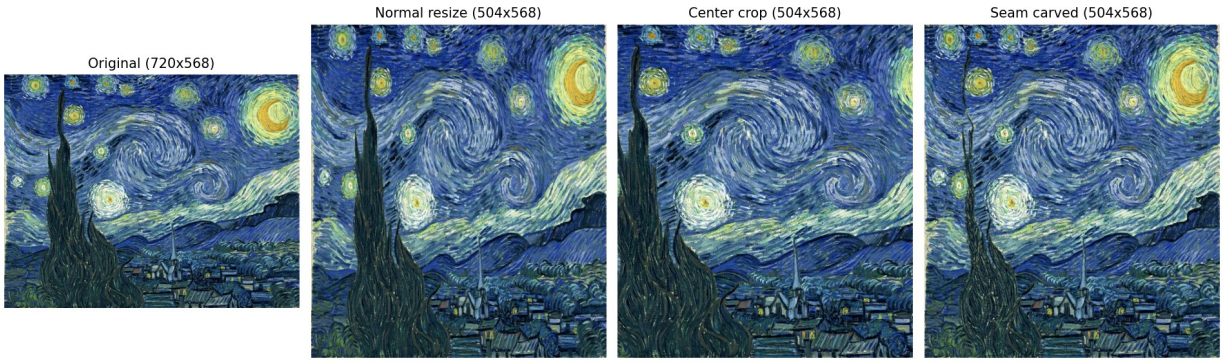


Figure 23: `starry_night.jpg`: $720 \times 568 \to 504 \times 568$. Every region of the painting is textured, so seam carving bends structures: the cypress is thinned and its outline becomes wavy.



Figure 24: Own photo with a person: $1000 \times 667 \to 700 \times 667$. The person keeps their height and proportions under seam carving; the resize squeezes the person, and the crop shifts the content.

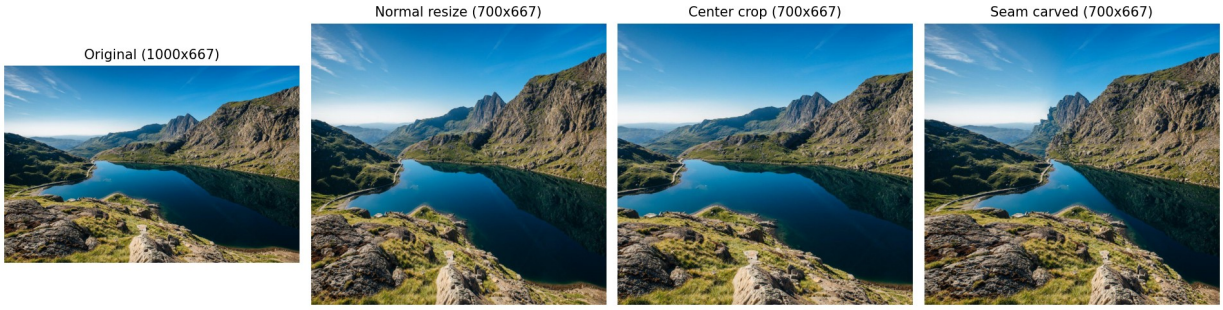


Figure 25: Own photo of a lake: $1000 \times 667 \to 700 \times 667$. Seam carving keeps the lake and the mountains in proportion, but the distant peak becomes jagged.

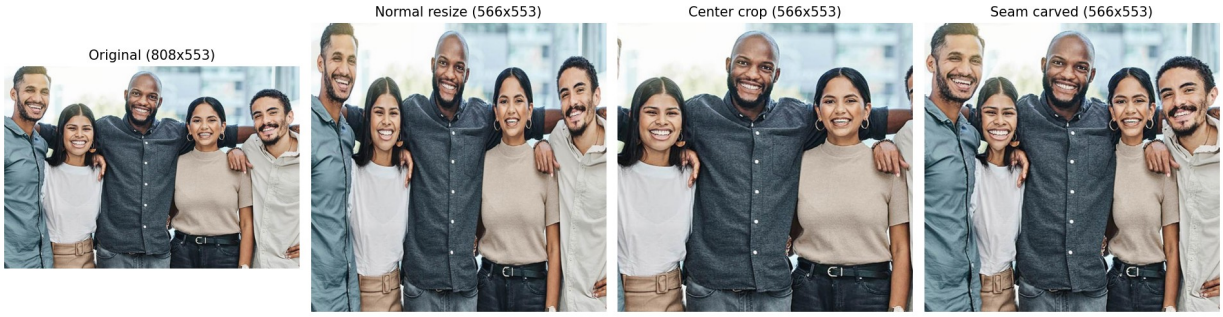


Figure 26: Own group photo (failure case): $808 \times 553 \to 566 \times 553$. The faces and bodies are narrowed, because there is almost no empty background to remove.

Summary. Seam carving beats the simple options when the image has large low-energy regions (sky, grass, fields, dark background): `rocket`, `home`, the person in the field and the lake. Resizing keeps all the content but squeezes every object, so people and towers look thinner. Cropping keeps the true proportions but throws content away (the outer towers of the rocket pad, the sides of the group photo). Seam carving is the only method that keeps both the content and the proportions in these images. It loses its advantage when texture or important content fills the whole width: `starry_night` and the group photo.

Two energy functions

We tried two energy functions on the group photo: the finite-difference gradient magnitude of Section 1.2, and the derivative-of-Gaussian (DoG) gradient magnitude of Section 1.3 with $\sigma = 2$. Both use reflection padding and the same dynamic program; only the energy map changes.



Figure 27: Group photo, 242 seams. Top row: original, finite-difference energy, finite-difference seams (red). Bottom row: DoG energy ($\sigma = 2$), DoG seams (cyan), and both seam sets together.

Question: how do the seams of the two energy functions differ? Because convolution with the Gaussian blurs the image before differentiating, the two energy maps look different. The finite-difference energy contains fine detail (hair, clothing texture and the background), so its low-energy paths are thin, jagged lines that wiggle through small gaps and are spread over the whole width of the image. The DoG energy keeps only the broad contours of the people (the outlines of heads, shoulders and arms) and has smooth, near-zero interiors. Its cheap paths are therefore wider corridors, and its seams gather into thicker bundles inside these smooth regions. The two seam sets are quite different: to compare them we took the sets of removed pixels in original coordinates (not seam by seam, since seams are numbered by removal order), and their Jaccard overlap is $70\,795/196\,857 = 36.0\,\%$. So only about a third of the removed pixels are shared, and the energy function clearly decides where the image loses its pixels. The price of the DoG energy is that the blur also spreads each edge over about σ pixels, so it localizes boundaries less precisely.

Failure case

Our designated failure case is the group photo of five people (and `starry_night` is a second example).

Question: what makes seam carving fail here?

- **Energy is not importance.** Gradient magnitude measures local intensity change; it does not know what a face is. Low-contrast regions such as skin, shirts and the gaps between bodies have low energy, so they look "cheap" even though they belong to the people we want to keep.
- **No empty columns.** The five people fill almost the whole frame, so every column crosses a person. Removing 30 % of the width (242 seams) is forced to pass through faces and bodies. The result narrows the faces and the gaps between people, although all five remain recognizable.
- **Dense texture** (`starry_night`). Every region has strong gradients, so there is no clearly cheap path. Seams choose the least bad ones and bend the structures, such as the cypress and the swirls.
- **Remedies.** Protect the important regions with a high-energy mask (for instance on detected faces), use a larger energy such as DoG or forward energy, or remove fewer seams and combine with cropping or resizing.

4 Bonus: Object Removal and Width Restoration

Seams can also make an object disappear, and inserted instead of removed they widen an image back to its size [4]. We remove an object from a photo with negative-energy seams, then grow the reduced photo back to its original width by inserting seams, and show both results together.

4.1 Removing an Object

What we did

- **Image.** A 1000×563 photo of a person (the "walker") standing on a grassy hill under a cloudy sky (Figure 28a).
- **Mask.** The mask is painted by clicking polygon vertices around the walker (`select_polygon_mask`; right-click finishes). `polygon_mask` rasterizes the vertices with `cv2.fillPoly` into a boolean array of the same size as the photo, and rejects fewer than three vertices, non-finite values and vertices outside the image. The final mask covers 2 733 pixels in a bounding box of 37×87 pixels (x from 481 to 517, y from 384 to 470); its widest row is 36 pixels wide (Figure 28b).
- **Energy.** The gradient-magnitude energy of Section 3.1 (`compute_energy`, reflection padding), copied to a `float64` array, with the energy of every masked pixel set to -10^6.
- **Removal loop** (`remove_masked_object`). Find the cheapest vertical seam with the dynamic program of Section 3.1, remove it from the photo *and from the mask*, recompute the energy on the reduced photo, and repeat until no masked pixel is left. The number of seams removed is k.

Code

```
def polygon_mask(shape, vertices):
    """Rasterize polygon vertices into a boolean mask matching (height, width)."""
    height, width = shape[:2]
    points = np.asarray(vertices, dtype=np.float64)
    if height < 1 or width < 1:
        raise ValueError("mask dimensions must be positive")
    if points.ndim != 2 or points.shape[1] != 2 or points.shape[0] < 3:
```

```
        raise ValueError("vertices must contain at least three (x, y) points")
    if not np.isfinite(points).all():
        raise ValueError("polygon vertices must be finite")
    if (
        np.any(points[:, 0] < 0)
        or np.any(points[:, 0] >= width)
        or np.any(points[:, 1] < 0)
        or np.any(points[:, 1] >= height)
    ):
        raise ValueError("polygon vertices must lie inside the image")
    mask = np.zeros((height, width), dtype=np.uint8)
    cv2.fillPoly(mask, [np.rint(points).astype(np.int32)], 1)
    return mask.astype(bool)


def remove_masked_object(image, mask, energy_function=compute_energy,
                         mask_energy=-1e6):
    """Remove masked pixels with negative-energy seams until the mask is empty."""
    image = _as_image(image)
    mask = np.asarray(mask, dtype=bool)
    if mask.shape != image.shape[:2]:
        raise ValueError("mask dimensions must match the image")
    if not np.any(mask):
        raise ValueError("mask must cover at least one pixel")
    if not np.isfinite(mask_energy) or mask_energy >= 0:
        raise ValueError("mask_energy must be a finite negative value")
    if not callable(energy_function):
        raise ValueError("energy_function must be callable")

    reduced = image.copy()
    remaining_mask = mask.copy()
    removed_seams = 0
    while np.any(remaining_mask):
        if reduced.shape[1] <= 1:
            raise ValueError("cannot remove remaining mask from a one-column image")
        energy = np.asarray(energy_function(reduced), dtype=np.float64).copy()
        if energy.shape != remaining_mask.shape or not np.isfinite(energy).all():
            raise ValueError("energy_function must return a finite image-sized map")
        energy[remaining_mask] = mask_energy
        seam = find_vertical_seam(energy)
        reduced = remove_vertical_seam(reduced, seam)
        remaining_mask = remove_vertical_seam(remaining_mask, seam).astype(bool)
        removed_seams += 1
    return reduced, removed_seams, remaining_mask
```

Why -10^6 makes the seam pass through the object

The energy of Section 1.2 is a gradient magnitude of an image with intensities in $[0, 1]$, so every entry is at most $\sqrt{2}$ and the natural cost of any seam is at most $563 \cdot \sqrt{2} \approx 800$. A seam that crosses p masked pixels costs about $-p \cdot 10^6$ plus that small natural part, so the dynamic program first maximizes the number of masked pixels it crosses and only then minimizes the natural energy. Masked pixels are therefore removed as fast as possible, while the rest of the seam still follows the cheap regions (sky and grass). The map has to be `float64`: next to values of order 10^6 the small natural differences would otherwise be lost in rounding.

Result

The mask was empty after $k = 36$ seams, so the reduced photo is 964×563 (Figure 28c). This is also the smallest possible number. A seam removes exactly one pixel per row, so a row of the mask that is w pixels wide needs at least w seams, and the widest row of our mask has $w = 36$.

The loop therefore needed no more seams than any method could, and the object is gone with only 36 columns of mostly sky removed.



(a) Original (1000 × 563). **(b)** Object mask (2 733 pixels). **(c)** Object removed (964 × 563, $k = 36$).

Figure 28: Bonus, removal stage. The mask in (b) is drawn at the same scale as the photo; (c) is shown at the same scale as (a), so it is slightly narrower.

4.2 Restoring the Width by Seam Insertion

What we did

- **Find the seams on a copy** (`find_insertion_locations`). Make a copy of the reduced photo and remove k seams from the copy, exactly as in Section 3.1. The copy is only used to find them.
- **Track the columns.** Alongside the copy we keep an array holding each pixel's column in the *reduced* photo and remove the same seams from it. Each seam is stored in these coordinates, as an array of $k \times H$ columns (one per row), so we know where every seam sat in the reduced photo.
- **Widen row by row** (`insert_vertical_seams`). Every pixel is written once, except the pixel of each of the k seams, which is written twice: the first copy is averaged with its left neighbor and the second with its right neighbor. At the border the pixel itself stands in for the missing neighbor. The neighbors are always taken from the reduced photo, never from pixels that were already inserted. The function checks that the seams are inside the image and unique within each row, and that every row ends with exactly $W + k$ pixels.
- **Record the output positions.** For each seam and row, the two output columns are stored (`inserted_positions`), and `draw_inserted_seams` paints both columns green for the figure.

Code

```python
def find_insertion_locations(image, seam_count, energy_function=compute_energy):
    """Find distinct seams on a shrinking copy and map them to source columns."""
    image = _as_image(image)
    height, width = image.shape[:2]
    if not isinstance(seam_count, (int, np.integer)) or seam_count < 0:
        raise ValueError("seam_count must be a non-negative integer")
    if seam_count >= width:
        raise ValueError("seam_count must be smaller than the image width")
    if not callable(energy_function):
        raise ValueError("energy_function must be callable")

    working_copy = image.copy()
    coordinate_map = np.tile(np.arange(width, dtype=np.int64), (height, 1))
    rows = np.arange(height)
    locations = []
    for _ in range(seam_count):
```

```
        energy = np.asarray(energy_function(working_copy), dtype=np.float64)
        if energy.shape != working_copy.shape[:2] or not np.isfinite(energy).all():
            raise ValueError("energy_function must return a finite image-sized map")
        seam = find_vertical_seam(energy)
        locations.append(coordinate_map[rows, seam].copy())
        working_copy = remove_vertical_seam(working_copy, seam)
        coordinate_map = remove_vertical_seam(coordinate_map, seam).astype(np.int64)
    if not locations:
        return np.empty((0, height), dtype=np.int64)
    return np.stack(locations)


def insert_vertical_seams(image, locations):
    """Insert two neighbor-averaged copies at each unique source seam pixel."""
    image = _as_image(image)
    height, width = image.shape[:2]
    locations = np.asarray(locations)
    if locations.ndim != 2 or not np.issubdtype(locations.dtype, np.integer):
        raise ValueError("locations must be a 2D integer array with one row per seam")
    if locations.shape[1] != height:
        raise ValueError("each seam must contain one source column per image row")
    if np.any(locations < 0) or np.any(locations >= width):
        raise ValueError("seam locations must lie inside the reduced image")

    seam_count = locations.shape[0]
    restored_shape = (height, width + seam_count) + image.shape[2:]
    restored = np.empty(restored_shape, dtype=np.float64)
    inserted_positions = np.empty((seam_count, height, 2), dtype=np.int64)

    for row in range(height):
        row_seam_order = np.argsort(locations[:, row], kind="stable")
        seam_columns = locations[row_seam_order, row]
        if np.unique(seam_columns).size != seam_count:
            raise ValueError("seam locations must be unique within each row")
        insertion_map = {
            int(column): int(seam_index)
            for column, seam_index in zip(seam_columns, row_seam_order)
        }
        output_column = 0
        for source_column in range(width):
            pixel = image[row, source_column]
            if source_column in insertion_map:
                seam_index = insertion_map[source_column]
                left = image[row, max(0, source_column - 1)]
                right = image[row, min(width - 1, source_column + 1)]
                inserted_positions[seam_index, row] = (
                    output_column,
                    output_column + 1,
                )
                restored[row, output_column] = 0.5 * (pixel + left)
                restored[row, output_column + 1] = 0.5 * (pixel + right)
                output_column += 2
            else:
                restored[row, output_column] = pixel
                output_column += 1
        if output_column != width + seam_count:
            raise RuntimeError("seam insertion produced an incorrect row width")
    return restored, inserted_positions


def restore_width_by_insertion(image, seam_count, energy_function=compute_energy):
    """Find seams on a copy, then insert them into the original reduced image."""
    image = _as_image(image)
    locations = find_insertion_locations(image, seam_count, energy_function)
```

```
    restored, inserted_positions = insert_vertical_seams(image, locations)
    return restored, locations, inserted_positions
```

Question: why do the k seams come from the copy, not from inserting the cheapest seam k times?

Suppose we inserted the currently cheapest seam, then recomputed the energy on the widened image, and repeated this k times. The two inserted columns are averages of a pixel with its neighbors, so they are smoother than their surroundings and have a lower gradient. The seam we just inserted (or one right next to it) is therefore again the cheapest one, and it would be chosen again and again. All k insertions would pile up in the same place, and that region would be stretched k times, which shows as a wide smeared band.

Taking the seams from a shrinking copy avoids this. After a seam is removed from the copy, the pixels it used are gone, so the next seam must use other pixels, and the k seams are pairwise different in every row. Mapping them back to the columns of the reduced photo through the tracked coordinates turns them into k different places where the photo can be widened by one pixel each, so the enlargement is spread over the image instead of being concentrated. The code relies on this property and checks that the seam columns are unique within each row.

4.3 Results

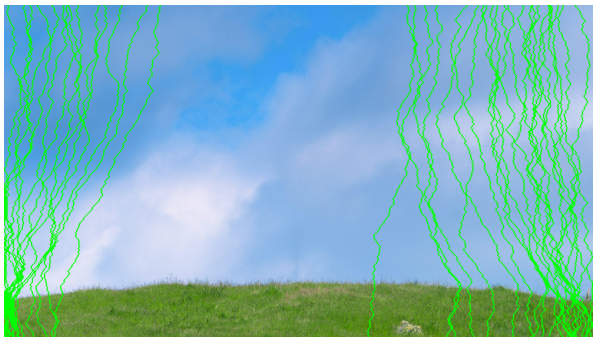
Figure 29 shows the photo with its mask, the photo without the object, and the photo back at its original width with the inserted seams drawn in green. Figure 30 shows the restored photo with and without the seams, and Figure 31 compares the walker's former position before and after.



Figure 29: Bonus result. Left: original with the object mask in red. Centre: object removed (964×563, 36 seams removed). Right: width restored to 1000×563; the 36 inserted seams are drawn in green.



(a) Restored photo (1000×563).



(b) The same photo with both columns of every inserted seam in green.

Figure 30: Bonus, restoration stage.



(a) Original.



(b) Restored.

Figure 31: Zoom on the walker's former position (the same pixel window, $x \in [400, 600)$, $y \in [330, 500)$, enlarged three times). The walker is gone and the grass line continues, but a faint vertical smudge remains in the sky where the walker stood.

Quantity	Value
Photo size	1000×563
Mask	2 733 pixels, bounding box 37×87, widest row 36
Seams removed to empty the mask (k)	36
Reduced size	964×563
Seams inserted	36
Restored size	1000×563
Green pixels in the overlay	$40\,536 = 36 \cdot 563 \cdot 2$

Table 5: Numbers of the Bonus experiment.

Observations

- **The object is removed and the width is restored.** The walker has disappeared, the horizon of the hill is continuous, and the restored photo has exactly the original size. Looking closely (Figure 31), a faint, thin vertical smudge is still visible in the sky above the grass where the walker stood; it is the one visible trace of the removal. The count of green pixels, $40\,536 = 36 \cdot 563 \cdot 2$, confirms that every seam wrote two columns in every row and that no seam overlapped another.
- **Where the seams went.** The inserted seams lie in the smooth sky and the grass at the left and right sides: none in the output columns 300 to 600, about 39 % of the inserted pixels in columns 0 to 300, and about 61 % in columns 600 to 1000. They touch 650 different output columns, so the widening is spread over a large part of the image as intended. In the lower-left corner many seams bunch up, because the grass and sky there are the cheapest region.
- **Cost.** Every seam costs $O(HW)$ as in Section 3.1: 36 seams for the removal and 36 for the copy, about $4 \cdot 10^7$ table entries in total for this photo.

Limitations

The restored photo is not the original photo without the walker, and the faint smudge described above shows that the removal is not perfectly clean. The large clouds are stretched slightly where seams were inserted, and the positions of objects change by a few pixels, because 36 columns were removed along the walker and 36 other columns were created elsewhere. The result looks

natural here because the photo consists mostly of smooth sky and grass: the object is small, there is plenty of low-energy content to hide the insertion, and the removal does not have to cross any strong edge. For an object on a textured background, or one much wider than 36 pixels, the same method would bend structures as in the failure cases of Section 3.2.

References

[1] A. Oliva, A. Torralba and P. G. Schyns. Hybrid images. *ACM Transactions on Graphics (SIGGRAPH)*, 25(3):527–532, 2006.

[2] P. J. Burt and E. H. Adelson. The Laplacian pyramid as a compact image code. *IEEE Transactions on Communications*, 31(4):532–540, 1983.

[3] P. J. Burt and E. H. Adelson. A multiresolution spline with application to image mosaics. *ACM Transactions on Graphics*, 2(4):217–236, 1983.

[4] S. Avidan and A. Shamir. Seam carving for content-aware image resizing. *ACM Transactions on Graphics (SIGGRAPH)*, 26(3):10, 2007.

A Implementation Notes

A.1 Padding Conventions

Part 1 uses zero padding, and Section 1.2 analyses the border lines it produces. Parts 2 and 3, and the Bonus (which reuses the Part 3 energy), use reflection padding (`padding_mode="reflect"` in `convolve_separable`) for the Gaussian, binomial and gradient filters: reflection mirrors the image content across the boundary, which eliminates the artificial intensity jump produced by zero padding and the border artefacts it causes in edge maps. This matters in particular for the seam-carving energy of Section 3.1, where a fake border edge would otherwise bias the seams.

A.2 Separable Convolution via SVD

`convolve_separable` (Section 1.1) detects rank-1 kernels by checking whether the ratio of the two largest singular values is below `rank_tolerance` $= 10^{-10}$. The Gaussian, binomial and derivative-of-Gaussian kernels used throughout the report are all rank-1 (outer products of 1D vectors), so the fast separable path, which costs $O(k)$ instead of $O(k^2)$ per pixel, is always taken for them; any non-separable kernel falls back to the vectorized version.

A.3 Binomial vs. Gaussian Filter in Pyramids

The pyramids of Section 2.3 use the binomial filter $[1, 4, 6, 4, 1]/16$ rather than a general Gaussian. Both are low-pass and approximately Gaussian; the binomial filter has integer-coefficient taps and is the efficient approximation used in the original Burt and Adelson construction [2].

A.4 Reproducibility

For Part 2, all output images and raw `.npy` arrays are saved under `outputs/`. Alignment points are cached in `outputs/part2_1/alignment_points.json`, so the results are fully reproducible without re-running the interactive alignment UI. For the Bonus, `save_bonus_outputs` writes the original, the mask preview, the binary mask, the object-removed photo, the restored photo and the restored photo with the green seams as PNG files, the mask, the reduced and restored arrays and the inserted-seam output positions as `.npy` files, and the three-panel `bonus_comparison.png`.